

Module 2 Lab Session 2: Unions, Generics, and Temporal

Field	Value
Module	M2 TypeScript 6.0 Essentials
Exercises	4 (AssessmentItem Union), 5 (Enrollment + Course Lifecycle), 6 (ApiResponse Generic), 7 (Temporal Timestamps)
After this session you can show	Discriminated unions with exhaustive never checks, a generic ApiResponse<T>, and UTC/timezone timestamps with Temporal.Instant replacing every use of legacy Date

Prerequisites Check Before You Start

Your tms-client project must contain the models/ folder with student.model.ts, course.model.ts, and enrollment.model.ts from Session 1, each with exported interfaces. If any are missing, go back and complete Session 1 the exercises below import those types.

Exercise 4: Assessment Types (Discriminated Unions)

The situation: The TMS has two types of graded items: quizzes and lab assignments. In M1, you modelled these with C# polymorphism an IGradable interface plus Quiz and LabAssignment classes. Each class had different properties but a shared CalculateGrade() method. TypeScript solves the same problem differently with a discriminated union.

The idea is familiar. The modelling tool is different:

- **C#:** One interface (IGradable) + two classes (Quiz, LabAssignment) + virtual dispatch at runtime.
- **TypeScript:** One union type (AssessmentItem) + a discriminant field (kind) + compiler-enforced narrowing.

Decision rule (use this in real projects):

- Use discriminated unions when the data comes from JSON (which has no class hierarchy) and you need exhaustive pattern matching.
- C# inheritance works well for shared runtime behaviour. TypeScript unions work well for data classification.

Trade-off you should know: Neither approach is universally better. C# inheritance lets you share implementation across subclasses. TypeScript unions make the compiler verify that you handle every variant.

Step 1 Define the Union

Create a new file `models/assessment.model.ts`:

```
export interface Quiz {  
  readonly id: string;  
  kind: "quiz";  
  title: string;  
  correctAnswers: number;  
  totalQuestions: number;  
}
```

```
export interface LabAssignment {  
  readonly id: string;  
  kind: "lab";  
  title: string;  
  functionalityScore: number;  
  codeQualityScore: number;  
}
```

```
export type AssessmentItem = Quiz | LabAssignment;
```

Step 2 Write the Grade Calculator

Add this function to the same file, below the type:

```
export function calculateGrade(item: AssessmentItem): number {  
  switch (item.kind) {  
    case "quiz":  
      return Math.round((item.correctAnswers / item.totalQuestions) * 100);  
    case "lab":  
      return Math.round(  
        item.functionalityScore * 0.7 + item.codeQualityScore * 0.3,  
      );  
  }  
}
```

Step 3 Test It

In `index.ts`, import and test:

```
import { AssessmentItem, calculateGrade } from "../models/assessment.model";
```

```
const quiz: AssessmentItem = {  
  id: "QUIZ-001",
```

```

kind: "quiz",
title: "SQL Basics",
correctAnswers: 8,
totalQuestions: 10,
};
const lab: AssessmentItem = {
  id: "LAB-001",
  kind: "lab",
  title: "REST API Project",
  functionalityScore: 85,
  codeQualityScore: 90,
};
console.log(`Quiz grade: ${calculateGrade(quiz)}%`); // 80
console.log(`Lab grade: ${calculateGrade(lab)}%`); // 87

// Verify readonly try this line and check the compiler error:
quiz.id = "QUIZ-999";
// ERROR: Cannot assign to 'id' because it is a read-only property

```

What you should see: Quiz prints 80%, lab prints 87%. The `quiz.id = "QUIZ-999"` line produces a compiler error. Inside the "quiz" case, try accessing `item.functionalityScore` the compiler reports an error because that property only exists on the "lab" variant.

Troubleshooting:

Problem	Likely cause	Fix
Property 'correctAnswers' does not exist outside the "quiz" case	TypeScript narrows the type inside each case block	This is correct behaviour access variant-specific properties only inside the matching case
No error on <code>quiz.id = "QUIZ-999"</code>	Missing <code>readonly</code> on the <code>id</code> property	Add <code>readonly</code> to <code>id</code> in both Quiz and LabAssignment interfaces
Grade calculation is NaN	<code>totalQuestions</code> is zero	Add a guard: <code>if (item.totalQuestions === 0) return 0;</code>

Exercise 5: Enrollment Lifecycle (State Machine Union)

The situation: The TMS tracks enrollment through 5 states: PENDING, APPROVED, ACTIVE, COMPLETED, DROPPED. The previous developer used 5 boolean flags creating 32 possible combinations, of which 27 are impossible.

```

// Legacy 27 impossible states allowed
interface EnrollmentBad {
  isPending: boolean;
  isApproved: boolean;
  isActive: boolean;
  isCompleted: boolean;
}

```

```
isDropped: boolean;
}
```

Can `isPending: true` and `isDropped: true` both be true at the same time? What would the UI show? A discriminated union makes these impossible states unrepresentable.

Decision rule (use this in real projects):

- Use a discriminated union whenever an entity moves through discrete states with different data per state.
- Each state variant should carry only the data that is meaningful for that state
PENDING needs the student and course IDs for the approval queue, COMPLETED needs the final grade for certificates, DROPPED needs the reason for reporting.

Trade-off you should know: Discriminated unions force you to handle every state explicitly. This is more code up front than boolean flags, but it eliminates an entire class of “impossible state” bugs. In Angular (M8–M9), you will use the same pattern for component state management.

Step 1 Model the Enrollment Lifecycle

Add this to `models/enrollment.model.ts` (below the existing `EnrollmentRecord` interface):

```
export type EnrollmentStatus =
| {
  status: "PENDING";
  requestedAt: Temporal.Instant;
  studentId: string;
  courseId: string;
}
| { status: "APPROVED"; approvedBy: string; approvedAt: Temporal.Instant }
| { status: "ACTIVE"; startDate: Temporal.PlainDate; currentGrade?: number }
| { status: "COMPLETED"; finalGrade: number; completedAt: Temporal.Instant }
| { status: "DROPPED"; reason: string; droppedAt: Temporal.Instant };
```

Step 2 Write the Exhaustive Handler

Add this to the same file:

```
export function describeEnrollment(enrollment: EnrollmentStatus): string {
  switch (enrollment.status) {
    case "PENDING":
      return `Awaiting approval since ${enrollment.requestedAt}`;
    case "APPROVED":
      return `Approved by ${enrollment.approvedBy}`;
    case "ACTIVE":
      return enrollment.currentGrade !== undefined
        ? `In progress grade so far: ${enrollment.currentGrade}`
        : `In progress not yet graded`;
  }
}
```

```

    case "COMPLETED":
      return `Finished with ${enrollment.finalGrade}`;
    case "DROPPED":
      return `Dropped: ${enrollment.reason}`;
    default: {
      const _check: never = enrollment;
      throw new Error(`Unhandled status: ${JSON.stringify(_check)}`);
    }
  }
}

```

Step 3 Test and Break It

```

const pending: EnrollmentStatus = {
  status: "PENDING",
  requestedAt: Temporal.Now.instant(),
  studentId: "STU-001",
  courseId: "CRS-101",
};
console.log(describeEnrollment(pending));
// Awaiting approval since 2026-05-08T...

```

Now break it: Comment out the DROPPED case and save. The compiler should immediately flag the default line because { status: "DROPPED"; ... } cannot be assigned to never.

What you should see: The code compiles cleanly with all 5 cases. Removing any case produces a compile error on the default line naming the missing variant.

Troubleshooting:

Problem	Likely cause	Fix
The never check does not trigger an error	You did not actually comment out a case	The error only appears when a union variant is unhandled
enrollment.approvedBy shows a type error outside the APPROVED case	TypeScript only allows access to state-specific fields inside the matching branch	This is correct move the access inside the APPROVED case

Exercise 5 Part B: Course Lifecycle

You just modelled how an enrollment moves through states. Courses have their own lifecycle a course starts as a draft, gets published for review, goes active when classes begin, and eventually gets archived or cancelled. The same discriminated union pattern applies.

Add the CourseStatus type to models/course.model.ts:

```
export type CourseStatus =
| { status: "DRAFT"; createdBy: string; createdAt: Temporal.Instant }
| { status: "PUBLISHED"; publishedAt: Temporal.Instant; syllabus: string }
| {
  status: "ACTIVE";
  enrolledCount: number;
  startDate: Temporal.PlainDate;
}
| {
  status: "ARCHIVED";
  archivedAt: Temporal.Instant;
  finalEnrollmentCount: number;
}
| { status: "CANCELLED"; reason: string; cancelledAt: Temporal.Instant };
```

Now write describeCourse yourself. Follow the same exhaustive switch pattern you used for describeEnrollment. Include the never check in the default case. The function signature is:

```
export function describeCourse(status: CourseStatus): string {
  // Your switch goes here. Handle all 5 states.
  // Each case should return a descriptive string using the state-specific fields.
  // Include the default/never check.
}
```

Test it in index.ts with an ACTIVE course:

```
import { CourseStatus, describeCourse } from "../models/course.model";
```

```
const webDev: CourseStatus = {
  status: "ACTIVE",
  enrolledCount: 28,
  startDate: Temporal.PlainDate.from("2026-09-01"),
};
console.log(describeCourse(webDev));
// Should print something like: Active with 28 students since 2026-09-01
```

Now break it: Comment out one of your cases. The compiler should flag the default line same pattern you just used in describeEnrollment.

If you are stuck: Look at your describeEnrollment function the structure is identical. Each case returns a string using the fields available in that specific variant.

Exercise 6: Reusable API Response (Generics)

The situation: Every TMS page fetches data from the API. Each fetch has the same lifecycle: loading, then either success with data or error with a message. Writing a separate response type for students, courses, and enrollments violates DRY.

In M1, you used C# generics like `List<Student>` and `Task<Student>`. TypeScript generics work the same way you write the wrapper once and fill in the data type when you use it.

Look at these two types. What is the same? What is different?

// Before generics repetitive

```
type StudentResponse =  
  | { status: "loading" }  
  | { status: "success"; data: Student; fetchedAt: Temporal.Instant }  
  | { status: "error"; message: string; statusCode: number };
```

```
type CourseListResponse =  
  | { status: "loading" }  
  | { status: "success"; data: Course[]; fetchedAt: Temporal.Instant }  
  | { status: "error"; message: string; statusCode: number };
```

The structure is identical. Only the data type changes. A generic eliminates the duplication.

Decision rule (use this in real projects):

- Use generics when you have a reusable wrapper that works with multiple data types (API responses, paginated lists, cache entries).
- Do NOT use generics when a function only ever works with one type.
`sortStudentsByGpa(students: Student[])` is clearer than `sort<T>(items: T[])` when `T` is always `Student`.

Trade-off you should know: C# generics survive to runtime the CLR knows the type and you can use `typeof(T)`. TypeScript generics are erased at compile time. The JavaScript output has no concept of `T`. If you need runtime type checking, combine generics with a type guard the generic handles compile-time safety, the guard handles runtime validation.

Step 1 Define the Generic

Create `models/api-response.model.ts`:

```
import { Temporal } from "@js-temporal/polyfill";
```

```
export type ApiResponse<T> =  
  | { status: "loading" }
```

```
| { status: "success"; data: T; fetchedAt: Temporal.Instant }  
| { status: "error"; message: string; statusCode: number };
```

Step 2 Write the Renderer

You have now written three exhaustive switch blocks (calculateGrade, describeEnrollment, describeCourse). This is the same pattern only the types are generic. Write the function body yourself:

```
export function renderResponse<T>(  
  response: ApiResponse<T>,  
  formatter: (data: T) => string,  
) : string {  
  // TODO: Handle all three states with a switch on response.status.  
  // "loading" → return "Loading..."  
  // "success" → call the formatter with response.data  
  // "error" → return a string with the statusCode and message  
}
```

If you are stuck: The switch body follows the exact same shape as describeEnrollment one case per union variant, return a string from each.

Step 3 Test with Different Data Types

In index.ts, import and test your renderResponse with two different data types:

```
import { ApiResponse, renderResponse } from "./models/api-response.model";  
import { Student } from "./models/student.model";  
import { Course } from "./models/course.model";
```

```
const studentRes: ApiResponse<Student> = {  
  status: "success",  
  data: {  
    id: "STU-001",  
    name: "Dawit Bekele",  
    enrollmentDate: Temporal.Now.instant(),  
    gpa: 3.4,  
  },  
  fetchedAt: Temporal.Now.instant(),  
};
```

```
console.log(  
  renderResponse(studentRes, (s) => `${s.name} GPA: ${s.gpa ?? "N/A"}`),  
);
```

// Now test with a different data type

```
const courseListRes: ApiResponse<Course[]> = {  
  status: "success",  
  data: [  

```



```

{
  id: "CRS-101",
  title: "Web Development Fundamentals",
  capacity: 30,
  startDate: Temporal.PlainDate.from("2026-09-01"),
},
],
fetchedAt: Temporal.Now.instant(),
};

console.log(
  renderResponse(courseListRes, (courses) =>
    courses.map((c) => c.title).join(", "),
  ),
);

```

What you should see: The first call prints Dawit Bekele GPA: 3.4. The second call prints Web Development Fundamentals. The same renderResponse function handles both ApiResponse<Student> and ApiResponse<Course[]> that is the power of the generic.

Troubleshooting:

Problem	Likely cause	Fix
Type 'Student' is not assignable to type 'T'	Missing type parameter on the variable	Specify the type: ApiResponse<Student>, not just ApiResponse
response.data is inaccessible	You are outside the "success" branch	data only exists on the success variant access it inside the case "success" block
formatter parameter type mismatch	Generic T not matching the data type	Ensure the formatter's parameter type matches what data holds

Exercise 7: Temporal Timestamps (Dates, Timezones, Durations)

The situation: The TMS needs precise timestamps. When was an enrollment approved? When does the assignment deadline expire? How many days until the course starts? The legacy code used JavaScript's Date object, where timezone math requires manual division by 1000 * 60 * 60 and the results vary across browsers.

In M1, you used DateTime.UtcNow for timestamps. The Temporal API is the ES2026 standard that replaces JavaScript's broken Date with immutable, timezone-aware types. Temporal.Instant maps directly to C#'s DateTime.UtcNow. Temporal.PlainDate maps to DateOnly.

Decision rule (use this in real projects):

- Use Temporal.Instant for recording when something happened (enrollment approval, grade submission). This is a single point on the UTC timeline.
- Use Temporal.PlainDate for dates without time (course start date, student birthday). No timezone ambiguity.
- Use Temporal.ZonedDateTime only for display converting a UTC instant to a user's local wall-clock time.

Trade-off you should know: TypeScript 6.0 gives you Temporal types at compile time. Whether Temporal works at runtime depends on your environment Node.js needs the @js-temporal/polyfill package, and browser support is rolling out (Chrome 144+, Firefox 139+). For the TMS, we include the polyfill and use Temporal as the standard. The Date object is not used in any new code.

Implementation

Add this to index.ts (the Temporal import should already be at the top from Session 1):

```
// 1. Record the exact moment an enrollment is approved (UTC)
const approvedAt = Temporal.Now.instant();
console.log(`Approved at (UTC): ${approvedAt}`);

// 2. Display in local timezone
const addisTime = approvedAt.toZonedDateTimeISO("Africa/Addis_Ababa");
const londonTime = approvedAt.toZonedDateTimeISO("Europe/London");
console.log(`Addis: ${addisTime.toPlainTime()}`);
console.log(`London: ${londonTime.toPlainTime()}`);
// Same moment, different wall-clock time

// 3. Course start date (date only, no time)
const courseStart = Temporal.PlainDate.from("2026-09-01");
const today = Temporal.Now.plainDateISO();
const daysUntilStart = today.until(courseStart).total({ unit: "days" });
console.log(`${Math.floor(daysUntilStart)} days until course starts`);

// 4. Assignment deadline duration
const deadline = Temporal.PlainDate.from("2026-12-15");
const remaining = today.until(deadline);
console.log(
  `${remaining.total({ unit: "days" })} days until assignment is due`
);
```

What you should see: All four sections print correct values. The Addis time is 3 hours ahead of London time. The “days until” calculations produce reasonable numbers based on today’s date.

Troubleshooting:

Problem	Likely cause	Fix
ReferenceError: Temporal is not defined	Missing import	Add import { Temporal } from "@js-temporal/polyfill"; at the top
TypeError: approvedAt.toZonedDateTimeISO is not a function	approvedAt is a string, not a Temporal.Instant	Make sure approvedAt comes from Temporal.Now.instant(), not a string
Negative "days until"	The target date is in the past	Update the date to a future value
Import error for @js-temporal/polyfill	Package not installed	Run npm install @js-temporal/polyfill

Module 2 What You Can Now Show

Confirm all seven:

- ☐ models/assessment.model.ts exports Quiz, LabAssignment, AssessmentItem, and calculateGrade
- ☐ calculateGrade correctly returns 80 for a quiz with 8/10 and 87 for a lab with 85 functionality + 90 quality
- ☐ Commenting out any case in describeEnrollment or describeCourse produces a compile error on the never default
- ☐ You wrote describeCourse yourself it was not copied from the handout
- ☐ renderResponse works with both ApiResponse<Student> and ApiResponse<Course[]> using the same function
- ☐ All Temporal timestamps use Temporal.Instant (not new Date()) zero instances of legacy Date in any model file
- ☐ Temporal.PlainDate.until() correctly calculates days between two dates

Module 2 What You Built

Review your tms-client project structure:

tms-client/

models/

student.model.ts Student + isStudent + parseStudent

course.model.ts Course + CourseStatus + describeCourse

enrollment.model.ts EnrollmentRecord + EnrollmentStatus + describeEnrollment

assessment.model.ts Quiz, LabAssignment, AssessmentItem + calculateGrade

api-response.model.ts ApiResponse<T> + renderResponse<T>

index.ts test code
tsconfig.json strict mode enabled

Every interface and function is exported and importable. In M8 you will move these models into the Angular project's `src/app/models/` folder. The patterns discriminated unions for state, generics for API responses, type guards for runtime validation are the same patterns Angular components and services use.